

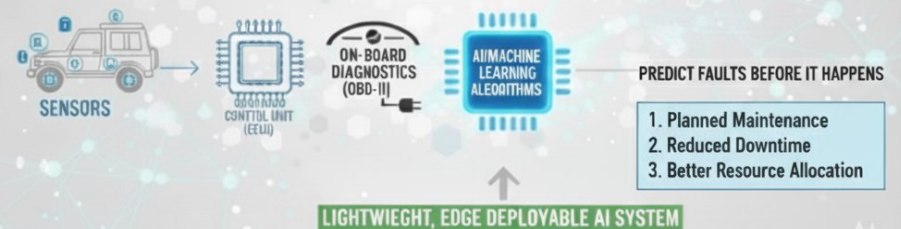
AI-DRIVEN PREDICTIVE MAINTENANCE FOR ARMY VEHICLES USING OBD-II SENSOR DATA

MODERN MILITARY VEHICLE MAINTENANCE STRATEGY

Reactive Maintenance (Current)



Predictive Maintenance (Future)

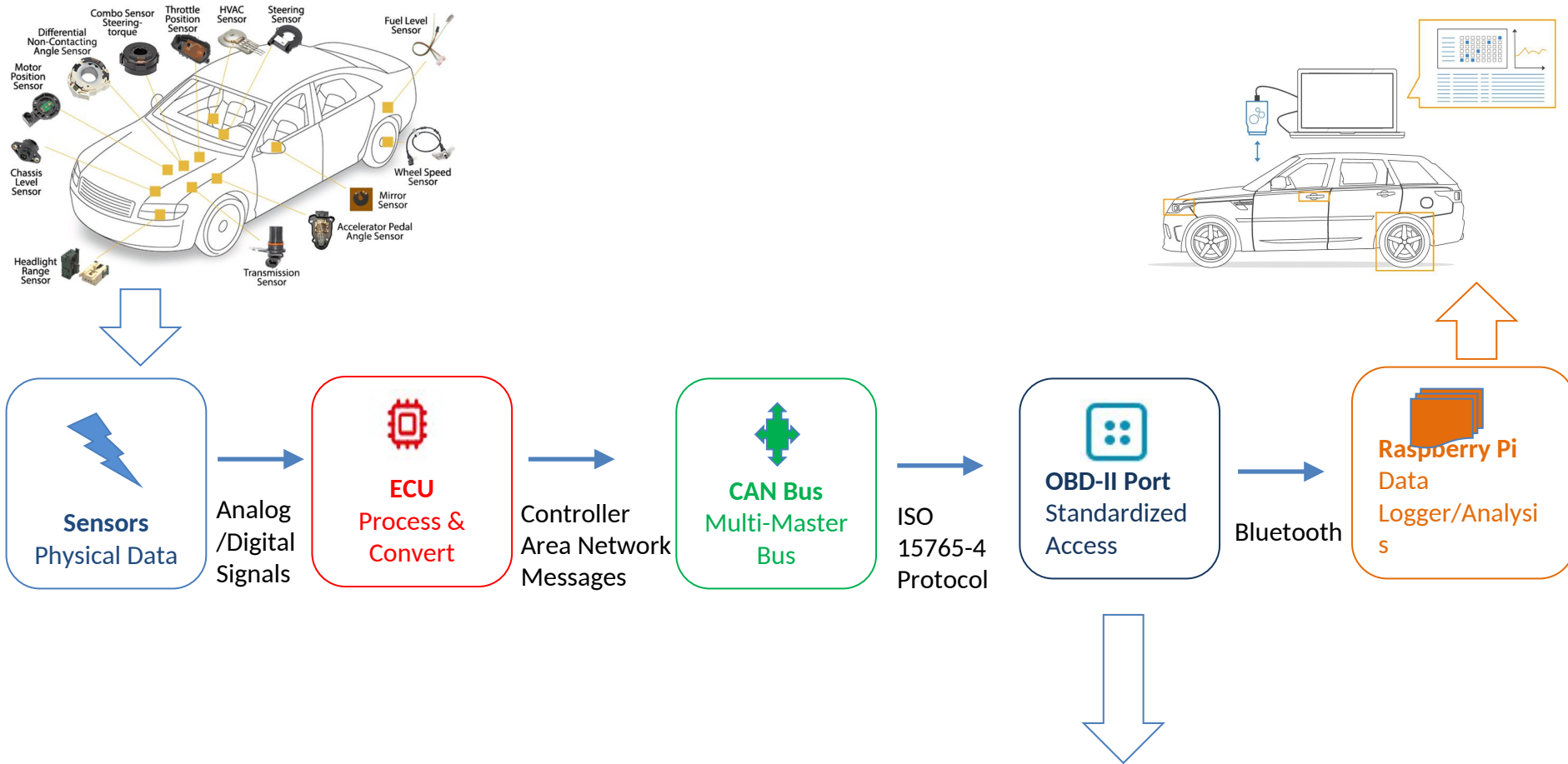


MTECH (AI) PROJECT – IISc Bangalore
Maj Vishvajeet Verma

PROBLEM STATEMENT AND MOTIVATION

- **Problem Statement**
 - Modern vehicles generate large amounts of sensor data through the **On-Board Diagnostics-II (OBD-II)** interface. (RPM, load, temp, throttle, etc.).
 - Current **maintenance practices are reactive** — once a Diagnostic Trouble Code (DTC) is triggered.
- **Key limitations of existing systems**
 - Threshold-based monitoring.
 - No predictive capability.
- **Goal: (Lightweight ,Edge Deployable AI-driven Predictive Maintenance (PdM) system)**
 - Uses live **OBD-II sensor data** to forecast faults **before** they occur.
- **Motivation**
 - Army vehicles operate in **harsh terrains** and **remote areas** where quick access to workshops or diagnostic tools is limited.
 - By predicting faults early, the system enables:
 - **Planned maintenance** during rest or refueling stops instead of roadside failures.
 - **Reduced downtime** and improved fleet availability.
 - **Better resource allocation** — spare parts and technician can be mobilized in advance.

OBD-II SYSTEM & DATA FLOW



OBD-II Dataset (Sample)

tstamp	RPM	Load(%)	CoolTemp(°C)	Speed(km/h)	Volt(V)	Lat	Long
2025-10-17 00:00	2150	45.2	87	62	13.8	13.0158	77.5656
2025-10-17 00:01	2180	47.5	88	63	13.9	13.0159	77.5657
2025-10-17 00:02	2220	48.1	89	65	13.7	13.0160	77.5658
2025-10-17 00:03	2280	50.0	90	67	13.8	13.0161	77.5659

FAULT FOCUS AND PID MAPPING

TARGET VEHICLE



Tata Safari GS-800

- 4-cylinder, ECU-controlled
- Widely used by Army — exposed to heat, dust, heavy load.
- High mechanical stress makes it an ideal platform for Predictive Maintenance (PDM).

Identified 5 Common Faults

1. Misfiring / Combustion Irregularity
2. Clogged Air Filter / Intake Restriction
3. Cooling System Fault / Overheating
4. Fuel System Inefficiency
5. Lubrication / Oil Pressure Problem

Selected 14 OBD-II PIDs (Sensor Parameters)

1. Engine RPM
2. Calculated Engine Load Value
3. Intake Manifold Absolute Pressure (MAP)
4. MAF Air Flow Rate
5. Calculated Instant Fuel Rate
6. Engine Coolant Temperature
7. Speed (GPS)
8. Fuel Rail Pressure
9. Power from MAF
10. OBD Module Voltage
11. Barometric Pressure
12. Ambient Air Temperature
13. Intake Air Temperature
14. Throttle Position (Manifold)

Rationale

- All faults can be **identified through existing onboard sensors**.
- The 14 PIDs cover all **key engine subsystems** — air, fuel, cooling, and lubrication and can be logged using OBD-II logger
- They provide **reliable, data-driven indicators** of early degradation.
- The scope is **practical and achievable** within project timelines.

EDGE SYSTEM DESIGN OVERVIEW

Layer 1: Data Capture

Real-time
OBD-II data
acquisition
from vehicle
ECU



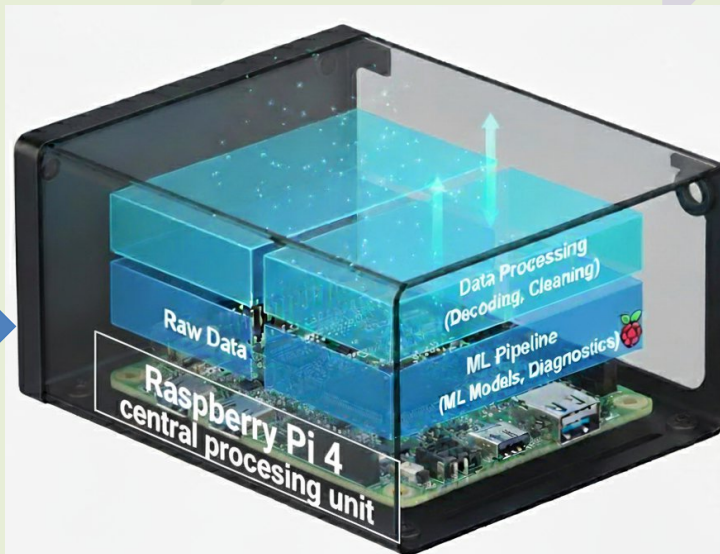
Raw
Data

Layer 2: Data Preprocess ing

Feature
engineering
and data
normalization

Layer 3: Model Inference

Edge ML
prediction
using
quantized
models



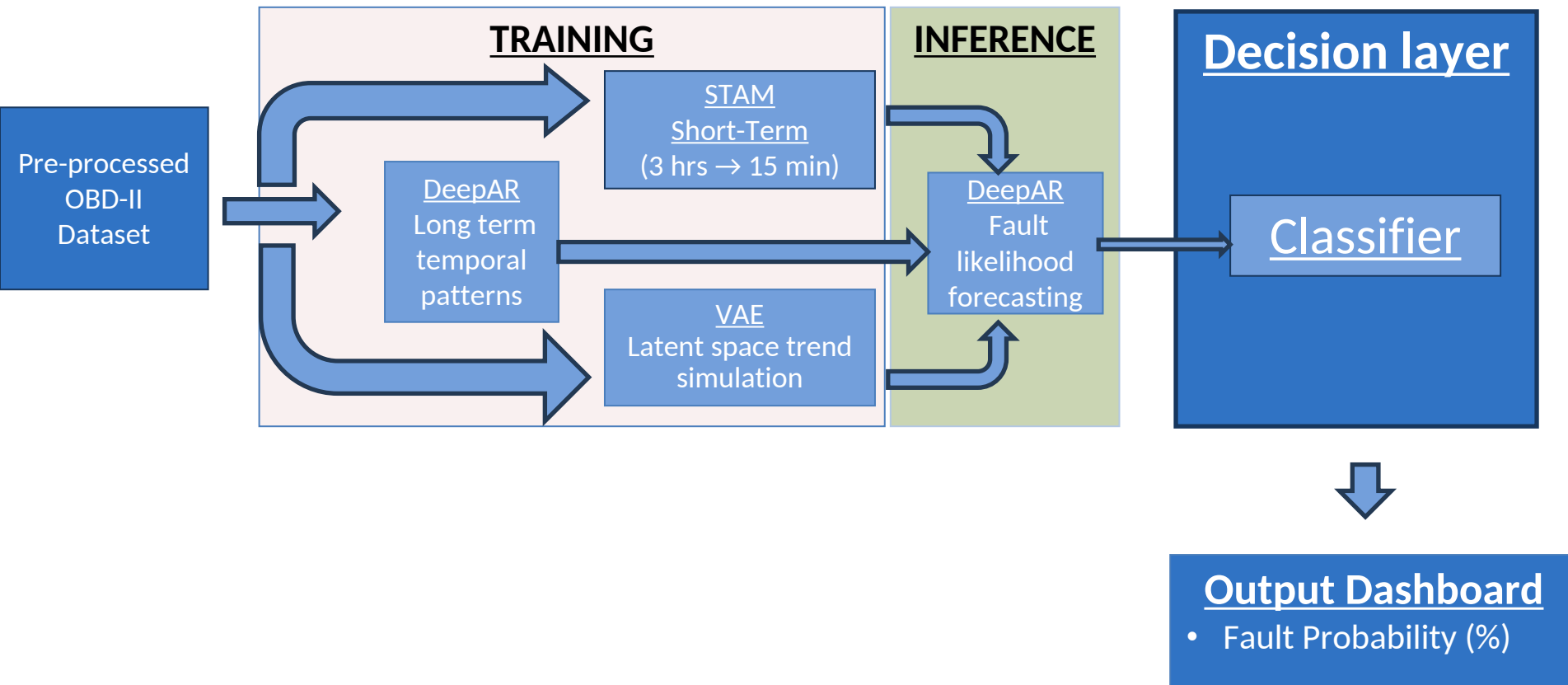
Final
Output

Layer 4: Dashboard & Alerts

Real-time
visualization
and
notification
system



PROPOSED METHODOLOGY



DeepAR- Deep Auto-regressive and recurrent NN
STAM- Spatio-Temporal Attention Mechanism
VAE- Variational Autoencoder

DeepAR- Training

(Teacher Forcing - No Autoregression)

Input window

Past Sensors

Past Faults

$t-3600 \rightarrow t-1801$



Future Sensors (Known)

Future Faults
(Ground Truth) (y_t)

$t-1800 \rightarrow t-1$

Total: 3600 sensor + 3600 fault
timesteps (all known)

Learns
patterns
from past
1800 steps

DeepAR
(LSTM/RNN)

Model Output

For each timestep, it
outputs mean (μ_t) and
standard deviation (σ_t)
— defining the
predicted probability
distribution of the fault

$(\mu, \sigma)[t-1800], (\mu, \sigma)[t-1799],$
 $(\mu, \sigma)[t-1798], \dots, (\mu, \sigma)[t-1]$

Loss Calculation

The true fault value (y_t) is
compared to this predicted
distribution using negative
log-likelihood(NLL)

$$\text{Loss} = -\log P(y_t | \mu_t, \sigma_t)$$

Key Points:

No Autoregression: Model has access to all true sensor and fault values

Teacher Forcing: Feeds true previous values as inputs to guide learning

Parallel Prediction: Produces all 1800 fault predictions simultaneously

Goal: Learn long-term temporal patterns from historical sequences

weights learned by
minimizing NLL are
used in inference
to produce future
means and variances
for unseen time
steps.

DeepAR- Inference(Autoregression)

Input

Past Sensors

Past Faults

$t-1800 \rightarrow t-1$

+

Future Sensors
(Unknown from STAM /VAE)

$t+1 \rightarrow t+1800$

Final Output

Complete sequence of 1800 predicted fault timesteps

Past Faults
[$t-1800:t-1$]
+ Future Sensor[1]

Predict Fault[1]

Past Faults
[$t-1799:t-1$]
+ **Fault[1]**
+ Future Sensor[2]

Predict Fault[2]

Past faults
[$t-1788:t-1$]
+ **Fault[1,2]** +
+ Future Sensor[3]

Predict Fault[3]

All predicted Faults
+
+ Future
Sensor[1800]

Predict
Fault[1800]

Autoregressive Prediction Loop

Why Autoregressive?

Each prediction depends on all previous predictions as Predicted fault at timestep t becomes input for $t+1$
The model's own outputs are fed back as inputs (auto-regression)
Enables sequential, one-step-at-a-time prediction capturing temporal dependencies

STAM (SPATIO-TEMPORAL ATTENTION MECHANISM)

Input

Past Sensor Data
 $X_{t-b:t-1} (\sim 3 \text{ hrs})$

Output

Predicted Future Sensors
 $\tilde{X}_{t:t+T} (\sim 30 \text{ mins})$

STAM ARCHITECTURE

Encoding

Encoder hidden states
 $h_t = \text{Encoder}(x_t, h_{t-1})$

Temporal Attention

$$e_{t,t'} = q_{t'}^T W_a h_t$$
$$\alpha_{t,t'} = \text{softmax}(e_{t,t'})$$
$$c_{t'} = \sum_t \alpha_{t,t'} h_t$$

Spatial Attention

$$\beta_{i,t'} = \text{softmax}(w_i^T c_{t'})$$
$$z_{t'} = \sum_i \beta_{i,t'} c_{t'}$$

Decoding And Loss

$$x_{\sim t'} = \text{Decoder}(z_{t'})$$
$$L = 1/T \sum_{i,t'} (x_{i,t'} - x_{\sim i,t'})^2$$

Key Points:

1. STAM uses these past 3 hours of data to **learn temporal and spatial correlations**:
 - Temporal**: How each sensor evolves over time (e.g., temperature trends)
 - Spatial**: How sensors influence each other (e.g., engine heat affecting oil pr.)
2. **Predicted Future Sensors** : Input (future sensors) to DeepAR for fault forecasting

VAE (VARIATIONAL AUTOENCODER)

Input

Out-of-sample vehicle sensor data

$$X'_{t'_0:t'_0+T}$$

D_{KL} term keeps the latent space close to a standard Normal distribution.

Output

Generate synthetic sensor data

$$\hat{X}_{t:t+T}$$

VAE ARCHITECTURE

Encoding

$$z = f_{\text{enc}}(X') = \mu z + \sigma z \odot \epsilon$$
$$\epsilon \sim N(0, I)$$

Latent Space

$z \in \mathbb{R}^d$ grouped into clusters via K Means

Decoding

$$\hat{X}_{t:t+T} = f_{\text{dec}}(z)$$

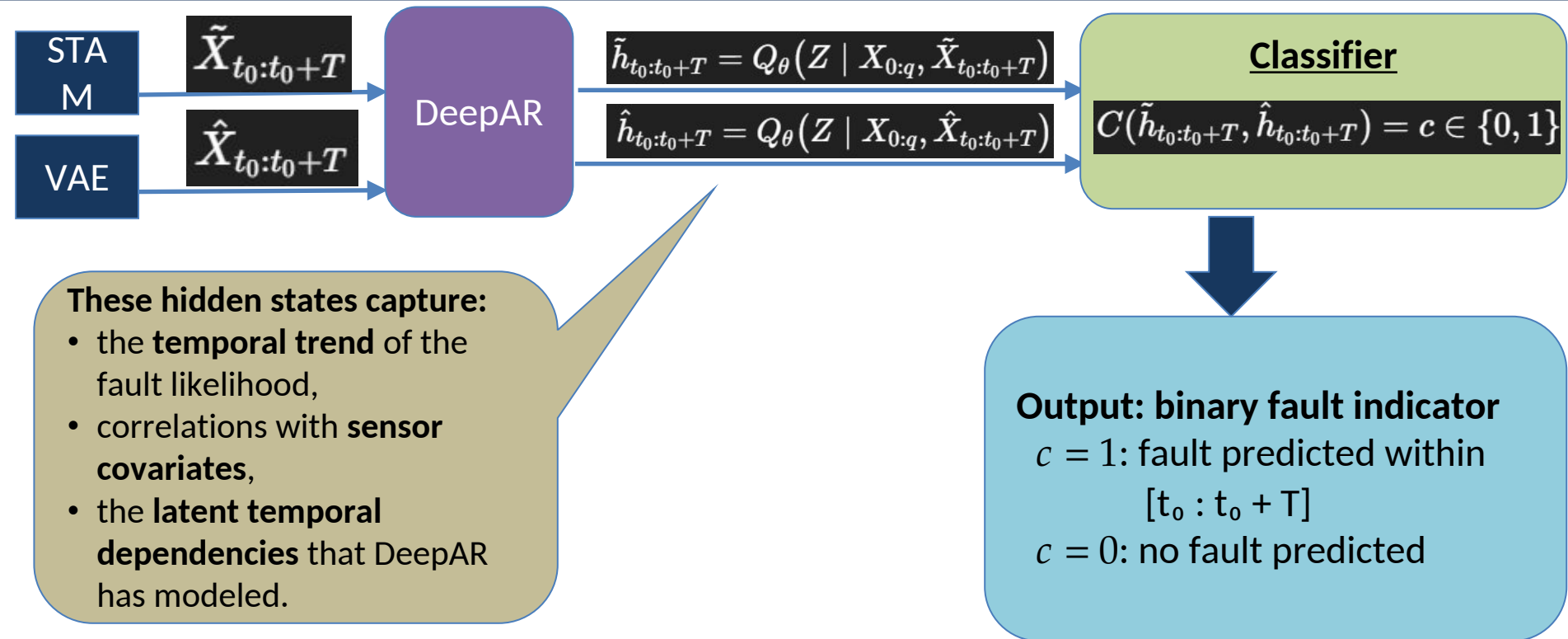
Reconstruction + Regularization loss

$$L = E_{q\phi}(z | X) [\| X - \hat{X} \|^2] + D_{KL}(q\phi(z | X) || p(z))$$

Key Points:

1. **Latent Representation:** Compresses high-dim sensor data into low-dim latent space z
2. **Clustering:** K-means groups similar vehicle states, enabling targeted synthetic data generation
3. **Generative Power:** Can sample and decode to create realistic sensor sequences for any operational state
4. Enables “**what-if**” **fault simulation** by sampling from different latent clusters.
5. Outputs used by **DeepAR** for probabilistic fault forecasting.

Classifier – Fault Occurrence Prediction



Key Points:

- DeepAR's hidden states encode future fault dynamics as high-level features.
- Classifier converts these hidden signals into binary fault occurrence.
- Uses both STAM and VAE generated sensor covariates for richer context.